

DBMS – SELLER AND INVENTORY MANAGEMENT

Project Title: EcoCart – Sustainable Products E-Commerce Order Management System

Skill Task: DBMS

Week: 11

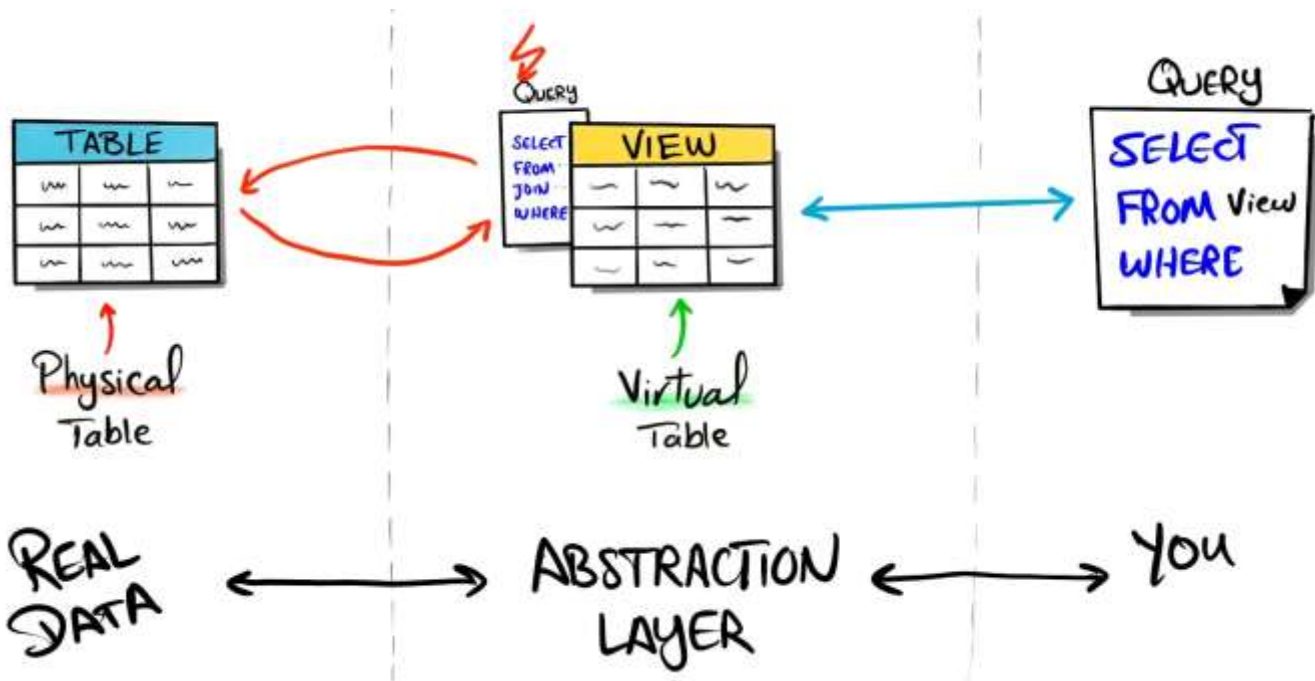
Database Automation using Views and Procedures

Objectives

1. Create views for frequently used reports.
2. Develop stored procedures for order processing.
3. Create functions for sales calculations.
4. Automate repetitive database operations.
5. Test database objects.

1. Create a View for Frequently Used Reports

A **view** is a virtual table based on a SQL query. It can be used to store frequently used reports.



Example: Customer Salary Report

Create View

```
SQL> CREATE OR REPLACE VIEW customer_report AS
 2  SELECT customer_id,
 3         customer_name,
 4         email,
 5         contact_no,
 6         address,
 7         registration_date
 8  FROM customer;
```

View created.

Display the View

```
SQL> SELECT * FROM customer_report;
```

CUSTOMER_ID

CUSTOMER_NAME

EMAIL

CONTACT_NO

ADDRESS

REGISTRAT

101

CUSTOMER_ID

CUSTOMER_NAME

EMAIL

CONTACT_NO

ADDRESS

REGISTRAT

2. Create a View for Customers by Address

```
SQL> CREATE OR REPLACE VIEW coimbatore_customers AS
  2  SELECT customer_id,
  3         customer_name,
  4         email,
  5         contact_no
  6  FROM customer
  7  WHERE address = 'Coimbatore';
```

View created.

```
SQL> SELECT * FROM coimbatore_customers;
```

CUSTOMER_ID

CUSTOMER_NAME

EMAIL

CONTACT_NO

101
Anitha Kumar
anitha@gmail.com
9876501234

CUSTOMER_ID

CUSTOMER_NAME

EMAIL

CONTACT_NO

103
Priya
priya@gmail.com
9876543212

3. Stored Procedure for Order Processing

For order processing, assume your project already has an ORDERS table.

SQL STORED PROCEDURES EXPLAINED

Reusable • Secure • Efficient

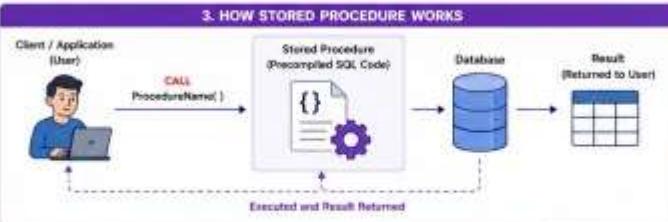
1. WHAT IS STORED PROCEDURE?

- A Stored Procedure is a precompiled collection of one or more SQL statements stored in the database.
- It can accept input, perform operations and return results.



2. WHY USE STORED PROCEDURES?

- Reusable SQL code
- Reduces code duplication
- Improves performance
- Enhances security
- Easier maintenance



3. HOW STORED PROCEDURE WORKS

6. EXAMPLE 1: SIMPLE STORED PROCEDURE

Table: Employee

EmpID	Employee	Department	Salary
101	Rahul	IT	50000
102	Aman	HR	45000
103	Neha	Sales	60000

Create Procedure:

```

DELIMITER $$
CREATE PROCEDURE GetEmpSalary()
BEGIN
    SELECT * FROM Employee;
END $$
DELIMITER ;

```

Example:

```

CALL GetEmpSalary();

```

Output:

EmpID	Employee	Department	Salary
101	Rahul	IT	50000
102	Aman	HR	45000
103	Neha	Sales	60000

7. EXAMPLE 2: STORED PROCEDURE WITH IN PARAMETER

Create Procedure:

```

DELIMITER $$
CREATE PROCEDURE GetEmpSalary(IN EmpID INT)
BEGIN
    SELECT * FROM Employee
    WHERE EmpID = EmpID;
END $$
DELIMITER ;

```

Example:

```

CALL GetEmpSalary(103);

```

Output:

EmpID	Employee	Department	Salary
103	Neha	Sales	60000

4. SYNTAX (MySQL)

```

DELIMITER $$
CREATE PROCEDURE ProcedureName(parameters)
BEGIN
    -- SQL Statements
END $$
DELIMITER ;

```

5. EXECUTE STORED PROCEDURE

```

CALL ProcedureName(parameters);

```

Example:

```

CALL GetEmpSalary();
CALL GetEmpSalary(103);

```

8. TYPES OF PARAMETERS

IN (Input)	OUT (Output)	INOUT (Input/Output)
Value is passed into procedure.	Value is returned from procedure.	Value is passed in and returned back.
Example: IN EmpID INT	Example: OUT Total INT	Example: INOUT Salary INT

9. CONTROL STATEMENTS IN STORED PROCEDURE

IF Statement

```

IF Salary > 50000 THEN
    SELECT 'High Salary';
ELSE
    SELECT 'Normal Salary';
END IF;

```

CASE Statement

```

CASE
    WHEN Salary > 50000 THEN 'Excellent'
    WHEN Salary > 30000 THEN 'Good'
    ELSE 'Average'
END CASE;

```

WHILE Loop

```

WHILE Counter <= 5 DO
    SET Counter = Counter + 1;
END WHILE;

```

REPEAT Loop

```

REPEAT
    SET Counter = Counter + 1;
UNTIL Counter > 5 END REPEAT;

```

10. DROP STORED PROCEDURE

Syntax:

```

DROP PROCEDURE ProcedureName;

```

Example:

```

DROP PROCEDURE GetEmpSalary;

```

11. STORED PROCEDURE vs FUNCTION

Feature	Stored Procedure	Function
Executed using	CALL statement	Used in SQL expression
Return	One or more result sets	Single value
Parameters	IN, OUT, INOUT	IN (only)
Can modify data	Yes	Depends (varies by DB)
Use case	Business operations	Calculations, validations

12. STORED PROCEDURE vs SQL QUERY

Feature	SQL Query	Stored Procedure
Execution	Write & run every time	Create once, use many times
Compilation	Compiled every time	Precompiled (faster)
Flexibility	Low	High
Maintenance	Difficult	Easy
Security	Less secure	More secure

13. ADVANTAGES

- Improves performance
- Reusable and modular code
- Reduces network traffic
- Enhances security
- Centralized business logic
- Easy maintenance

14. DISADVANTAGES

- Complex procedures are hard to debug
- Not portable across databases
- Needs proper permissions
- Can become large and hard to maintain

15. REAL-WORLD USE CASES

- Banking - Balance check, Money transfer
- E-Commerce - Place order, Update stock
- Hospital - Patient registration, Reports
- School - Result processing, Reports
- HR System - Salary processing, Reports



BEST PRACTICES

- Use meaningful names
- Keep procedures short and focused
- Validate input parameters

- Use proper error handling
- Add comments for better understanding
- Grant execute permission instead of table access



COMMON MISTAKES

- Forgetting CALL keyword
- Incorrect parameter order
- Not using delimiters in MySQL



MEMORY TRICK

WRITE ONCE → STORE IN DB → CALL MANY TIMES
= STORED PROCEDURE

```

SQL> CREATE OR REPLACE PROCEDURE add_order(
2     p_order_id NUMBER,
3     p_customer_id NUMBER,
4     p_total_amount NUMBER
5 )
6 IS
7 BEGIN
8
9     INSERT INTO orders
10        (order_id, customer_id, order_date, total_amount)
11    VALUES
12        (p_order_id, p_customer_id, SYSDATE, p_total_amount);
13
14    DBMS_OUTPUT.PUT_LINE('Order added successfully');
15
16 END;
17 /

```

Procedure created.

Execute the Procedure

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> EXEC add_order(101, 101, 5000);
Order added successfully

PL/SQL procedure successfully completed.
```

4. Function for Sales Calculation

A function can calculate the total sales amount.

SQL FUNCTIONS EXPLAINED

Reusable • Efficient • Return One Value

1. WHAT IS A FUNCTION?

- A Function is a reusable database object.
- It accepts input (parameters), performs a specific task and returns a single value.
- Functions can be used inside SQL statements like SELECT, WHERE, ORDER BY etc.

2. WHY USE FUNCTIONS?

- Code reusability
- Reduces repeated calculations
- Cleaner and shorter SQL queries
- Easy maintenance
- Improves readability

5. EXAMPLE 1: SIMPLE FUNCTION

Function: Calculate Annual Salary (Monthly Salary * 12)

```
CREATE FUNCTION AnnualSalary(MonthlySalary INT)
RETURNS INT
BEGIN
    RETURN MonthlySalary * 12;
END;
```

Use Function:

```
SELECT AnnualSalary(MHRE) AS AnnualSalary;
Output: AnnualSalary
        480000
```

3. HOW SQL FUNCTION WORKS

Input (Parameters) → **FUNCTION** (Processing / Logic) → Return (Single Value)

e.g. 50000 → e.g. 600000

Used in SQL Statements: SELECT, WHERE, ORDER BY

Function always returns one value using RETURN statement.

6. EXAMPLE 2: FUNCTION WITH TABLE

EmpID	EmpName	Salary
101	Rahul	50000
102	Aman	45000
103	Neha	60000

Query Using Function:

```
SELECT EmpName, Salary,
       AnnualSalary(Salary) AS AnnualSalary
FROM Employee;
```

EmpName	Salary	AnnualSalary
Rahul	50000	600000
Aman	45000	540000
Neha	60000	720000

How it works?

For each row → AnnualSalary(Salary) is called and returns Salary * 12

4. SYNTAX (MySQL)

```
DELIMITER $$
CREATE FUNCTION FunctionName(parameter datatype)
RETURNS return_datatype
BEGIN
    -- SQL Statements
    RETURN value;
END $$
Reset delimiter: DELIMITER ;
```

7. TYPES OF FUNCTIONS

A. BUILT-IN FUNCTIONS (Already provided by DBMS)

- Aggregate: COUNT(), SUM(), AVG(), MAX(), MIN()
- String: UPPER(), LOWER(), CONCAT(), LENGTH(), SUBSTRING()
- Numeric: ROUND(), CEIL(), FLOOR(), ABS(), MOD()
- Date/Time: NOW(), CURRENT(), YEAR(), MONTH(), DAY()

No need to create. Use directly in queries.

B. USER DEFINED FUNCTIONS (UDF)

Created by users for custom logic.

```
CREATE FUNCTION Bonus(Salary INT)
RETURNS INT
BEGIN
    RETURN Salary * 0.10;
END;
```

Need to create using CREATE FUNCTION.

8. RETURN STATEMENT

- RETURN is mandatory in a function.
- It sends the result back to the caller.
- Without RETURN, function will not return any value.

Example:

```
RETURN MonthlySalary * 12;
```

9. FUNCTION vs STORED PROCEDURE

Feature	Function	Stored Procedure
Returns	Always returns one value	May return zero, one or many values
Usage	Used in SQL query (SELECT, WHERE)	Executed using CALL statement
Purpose	Calculations, formatting	Business logic, operations
Can modify data	Generally No (in most DBMS)	Yes
Can return result set	No	Yes
Parameters	At parameters only	IN, OUT, INOUT
Example	SELECT UPPER('rahul');	CALL SetEmpSalary();

10. EXAMPLE: USER DEFINED FUNCTION

Calculate Bonus (10% of Salary)

```
CREATE FUNCTION Bonus(Salary INT)
RETURNS DECIMAL(10,2)
BEGIN
    RETURN Salary * 0.10;
END;
```

Use Function:

```
SELECT EmpName, Salary, BonusSalary AS Bonus
FROM Employee;
```

EmpName	Salary	Bonus
Rahul	50000	5000.00
Aman	45000	4500.00
Neha	60000	6000.00

Execution Flow:

```
Input Parameter (Salary)
↓
Processing Salary * 0.10
↓
RETURN (Bonus Amount)
```

11. COMMON BUILT-IN FUNCTIONS EXAMPLES

```
COUNT() SELECT COUNT(*) FROM Employee;
SUM() SELECT SUM(Salary) FROM Employee;
AVG() SELECT AVG(Salary) FROM Employee;
UPPER() SELECT UPPER('rahul'); -- RAHUL
LOWER() SELECT LOWER('RAHUL'); -- rahul
LENGTH() SELECT LENGTH('Rahul'); -- 5
NOW() SELECT NOW(); -- Current Date & Time
YEAR() SELECT YEAR('2020-05-20'); -- 2020
ROUND() SELECT ROUND(122.456,2); -- 122.46
```

12. REAL-WORLD APPLICATIONS

- Banking: Interest calculation
- E-Commerce: Discount calculation
- School: Percentage calculation
- Hospital: Age calculation
- HR Department: Annual salary, Bonus calculation
- GST Billing: Tax calculation

13. ADVANTAGES

- Reusability of code
- Reduces repeated calculations
- Makes queries shorter and cleaner
- Easy to maintain
- Improves readability and productivity

14. DISADVANTAGES

- Complex functions may affect performance
- Not suitable for complex operations
- Database specific (portability issue)
- Debugging can be difficult

15. BEST PRACTICES

- Use meaningful function names
- Keep function small and focused
- Provide input parameters
- Always use RETURN with correct data type
- Avoid complex business logic in functions

16. COMMON MISTAKES

- Forgetting RETURN statement
- Returning wrong data type
- Incorrect number of parameters
- Using function for complex operations

MEMORY TRICK

Input → Function → Process → RETURN → One Output

```
SQL> CREATE OR REPLACE FUNCTION calculate_sales(  
2     p_quantity NUMBER,  
3     p_price NUMBER  
4 )  
5 RETURN NUMBER  
6 IS  
7     v_total NUMBER;  
8 BEGIN  
9  
10    v_total := p_quantity * p_price;  
11  
12    RETURN v_total;  
13  
14 END;  
15 /
```

Function created.

Execute Function

```
SQL> SELECT calculate_sales(5, 1000) AS total_sales  
2 FROM dual;  
  
TOTAL_SALES  
-----  
5000
```

5. Procedure to Automate Order Status

Instead of manually changing every order, a procedure can update them automatically.

```
SQL> CREATE OR REPLACE PROCEDURE update_order_amount  
2 IS  
3 BEGIN  
4  
5     UPDATE orders  
6     SET total_amount = total_amount * 0.90  
7     WHERE total_amount > 10000;  
8  
9     DBMS_OUTPUT.PUT_LINE('Order amounts updated successfully');  
10  
11 END;  
12 /
```

Procedure created.

Execute

```
SQL> EXEC update_order_amount;  
Order amounts updated successfully  
  
PL/SQL procedure successfully completed.
```